Real-Time Activity Detection Using phyphox Motion Data

A practical setup guide for streaming phone motion sensors over local Wi-Fi and running an activity detector AI model on a laptop.

Component	Role
Phone + phyphox	Streams accelerometer / linear acceleration / gyroscope data
Local Wi-Fi	Connects phone and laptop on the same network
Laptop Python script	Reads phyphox REST API, buffers sensor windows, runs inference
Activity model	Predicts labels such as sitting, walking, running, stairs, etc.

1. Overall Pipeline

The real-time testing loop is simple:

- Phone runs phyphox and collects motion sensor data.
- Laptop connects to the phyphox Remote Access URL over the same Wi-Fi.
- Python repeatedly requests new sensor samples from the phyphox /get endpoint.
- The script keeps a sliding time window, for example 2 seconds.
- The window is preprocessed exactly like your training data.
- Your trained activity detector outputs a live activity label.

2. Start phyphox on the Phone

- Connect the phone and laptop to the same local Wi-Fi network.
- Open phyphox on the phone.
- Open an experiment such as Linear Acceleration or Acceleration with g.
- Enable Remote Access from the phyphox menu.
- Copy the address shown by phyphox, for example http://192.168.0.42:8080.

For activity detection, Linear Acceleration is usually the best first choice because it reduces the gravity component compared with raw acceleration.

3. Check Buffer Names

Different phyphox experiments may use different internal buffer names. Open this address in the laptop browser:

```
http://PHONE_IP:PORT/config
```

Look for names such as t, x, y, z or acc_time, accX, accY, accZ. Use the exact names in the Python script.

4. Install Python Dependencies

```
pip install requests numpy scipy joblib
```

If your model is PyTorch-based, also install torch. If it is TensorFlow/Keras-based, install tensorflow.

5. Real-Time Streaming and Inference Script

Replace PHYPOX_URL and the buffer names after checking /config. This example assumes a scikit-learn model saved as activity_model.pkl.

```
import time
import requests
import numpy as np
from collections import deque
import joblib

# Change this to the address shown by phyphox
PHYPOX_URL = "http://192.168.0.42:8080"

# Change these after checking http://PHONE_IP:PORT/config
TIME_BUF = "t"
X_BUF = "x"
Y_BUF = "y"
Z_BUF = "z"

# Load your trained activity detector
model = joblib.load("activity_model.pkl")

WINDOW_SECONDS = 2.0
POLL_INTERVAL = 0.1

buffer = deque()
last_t = 0.0


def fetch_new_samples(last_t):
    """
    Fetch only samples newer than last_t.
    The %7C symbol is encoded '|'. It means: align x/y/z values
    with the time buffer threshold.
    """
    url = (
        f"{PHYPOX_URL}/get?"
        f"{TIME_BUF}={last_t}"
        f"&{X_BUF}={last_t}%7C{TIME_BUF}"
        f"&{Y_BUF}={last_t}%7C{TIME_BUF}"
        f"&{Z_BUF}={last_t}%7C{TIME_BUF}"
    )

    r = requests.get(url, timeout=1.0)
    r.raise_for_status()
    data = r.json()

    t = data["buffer"][TIME_BUF]["buffer"]
    x = data["buffer"][X_BUF]["buffer"]
    y = data["buffer"][Y_BUF]["buffer"]
    z = data["buffer"][Z_BUF]["buffer"]

    return list(zip(t, x, y, z))


def extract_features(window_array):
    """
    Replace this with exactly the same preprocessing/features
    used during model training.
    window_array shape: [N, 4] = time, x, y, z
    """
    acc = window_array[:, 1:4]
    mag = np.linalg.norm(acc, axis=1)

    features = [
        acc[:, 0].mean(), acc[:, 1].mean(), acc[:, 2].mean(),
        acc[:, 0].std(),  acc[:, 1].std(),  acc[:, 2].std(),
        mag.mean(), mag.std(), mag.max(), mag.min(),
    ]

    return np.array(features).reshape(1, -1)


print("Streaming started...")

while True:
    try:
        samples = fetch_new_samples(last_t)

        if samples:
            for sample in samples:
                buffer.append(sample)

            last_t = samples[-1][0]

            # Keep only a recent sliding window
            while buffer and buffer[-1][0] - buffer[0][0] > WINDOW_SECONDS:
                buffer.popleft()
```

```
        window = np.array(buffer)

        if len(window) > 10:
            X = extract_features(window)
            pred = model.predict(X)[0]

            if hasattr(model, "predict_proba"):
                conf = model.predict_proba(X).max()
                print(f"Activity: {pred} | confidence: {conf:.2f}")
            else:
                print(f"Activity: {pred}")

    time.sleep(POLL_INTERVAL)

except requests.exceptions.RequestException as e:
    print("Connection error:", e)
    time.sleep(1)

except KeyboardInterrupt:
    print("Stopped.")
    break
```

6. If Your Model Uses Raw Windows

If your model was trained directly on raw x, y, z windows rather than handcrafted features, replace the feature extraction call with a raw-window tensor/array.

```
raw = window[:, 1:4]        # x, y, z only
raw = raw.reshape(1, raw.shape[0], raw.shape[1])
pred = model.predict(raw)
```

7. PyTorch Inference Example

```
import torch

model.eval()

raw = window[:, 1:4].astype(np.float32)
x_tensor = torch.tensor(raw).unsqueeze(0)  # [1, time, channels]

with torch.no_grad():
    logits = model(x_tensor)
    pred_id = torch.argmax(logits, dim=1).item()
```

8. Prediction Smoothing

Real-time motion predictions can jump. A simple majority vote over the last few predictions can make output more stable.

```
from collections import deque

last_predictions = deque(maxlen=5)

last_predictions.append(pred)
smoothed_pred = max(set(last_predictions), key=last_predictions.count)
print("Smoothed activity:", smoothed_pred)
```

9. Practical Testing Protocol

- First test one class at a time: sitting, standing, walking, running.
- Hold the phone in the same position/orientation used during training.
- Use the same sampling rate and window length as training when possible.
- Print both raw prediction and smoothed prediction.
- Record a short CSV log during the test so you can debug false predictions later.

10. Common Problems and Fixes

Problem	Likely Cause	Fix
KeyError in Python	Wrong buffer names	Open /config and copy the exact buffer names
No connection	Phone and laptop not on same Wi-Fi or firewall blocking access	Check Wi-Fi, IP address, port, and browser access first
Delayed predictions	Polling too slow or too much data requested	Use timestamp threshold requests and avoid full-buffer requests
Unstable labels	Short window or noisy sensor signal	Use 2-3 second window and majority-vote smoothing
Wrong activity	Train/test mismatch	Match phone position, sensor type, preprocessing, and window size

11. Final Recommendation

Start with Linear Acceleration x, y, z; use a 2-second sliding window; poll every 0.1 seconds; and keep preprocessing identical to training. Once the loop works, add gyroscope channels and confidence smoothing.

References

- phyphox Remote-interface communication: https://phyphox.org/wiki/index.php/Remote-interface_communication
- phyphox Network Connections documentation: https://phyphox.org/wiki/index.php/Network_Connections
- phyphox main website: https://phyphox.org/